r3st0.fr

Documentation technique complète

Sékou Mahamadou Zan DIARRA

BTS SIO · SLAM — Projet pédagogique PHP MVC + MySQL

Stack : PHP · MVC · MySQL · HTML/CSS · WampServer

1. Présentation du projet

r3st0.fr est un site de critiques de restaurants développé dans le cadre d'un projet pédagogique de BTS SIO option SLAM. L'objectif principal est de découvrir et mettre en pratique le pattern MVC (Modèle-Vue-Contrôleur) en PHP tout en manipulant une base de données MySQL.

Le site propose une expérience similaire à TripAdvisor ou LaFourchette : les utilisateurs peuvent consulter des restaurants, lire des avis, s'inscrire, noter et commenter les établissements.

1.1 Objectifs pédagogiques

- Comprendre et implémenter le pattern MVC en PHP natif
- Manipuler une base de données MySQL via PDO
- Gérer l'authentification par sessions PHP
- Concevoir des formulaires HTML avec traitements GET/POST
- Organiser un projet web en couches séparées

1.2 Profils utilisateurs

Visiteur non connecté

- Consulter la liste de tous les restaurants
- Voir la fiche détail d'un restaurant (photos, description, horaires, avis)
- Effectuer une recherche par nom, adresse ou type de cuisine
- Accéder au formulaire de connexion / inscription

Utilisateur connecté

- Toutes les fonctionnalités visiteur
- Marquer un restaurant comme « aimé »
- Poster un avis (note 1-5 + commentaire texte)
- Supprimer son propre avis

2. Architecture — Pattern MVC

Le projet suit strictement le pattern MVC. Le point d'entrée unique est index.php qui lit le paramètre GET action et délègue au bon contrôleur via la fonction controleurPrincipal().

2.1 Structure des dossiers

```
r3st0_2024_ressources/
├── base de donnees/
│   └── resto.sql              # Dump complet de la base MySQL
└── site/
    ├── index.php              # Point d'entrée unique
    ├── getRacine.php          # Utilitaire : chemin absolu racine
    ├── controleur/            # Couche Contrôleur
    │   ├── controleurPrincipal.php
    │   ├── listeRestos.php
    │   ├── detailResto.php
    │   ├── rechercheResto.php
    │   ├── connexion.php
    │   ├── deconnexion.php
    │   ├── aimer.php
    │   └── critiquer.php
    ├── modele/                # Couche Modèle
    │   ├── bd.pdo.php
    │   ├── authentification.php
    │   ├── bd.resto.php
    │   ├── bd.photo.php
    │   ├── bd.typecuisine.php
    │   ├── bd.utilisateur.php
    │   ├── bd.aimer.php
    │   └── bd.critiquer.php
    ├── vue/                   # Couche Vue
    │   ├── entete.html.php
    │   ├── pied.html.php
    │   ├── vueListeRestos.php
    │   ├── vueDetailResto.php
    │   ├── vueRechercheResto.php
    │   ├── vueAuthentification.php
    │   └── vueConfirmation.php
    ├── css/                   # Feuilles de style
    ├── images/                # Icônes UI
    └── photos/                # Photos des restaurants
```

2.2 Tableau récapitulatif MVC

Couche	Fichiers	Rôle
Modèle	`modele/bd.*.php`	Accès BDD, requêtes SQL, fonctions métier

Vue	`vue/vue*.php`	Génération du HTML, affichage des données
Contrôleur	`controleur/*.php`	Orchestration : reçoit la requête, appelle le modèle, inclut la vue

3. Flux d'exécution — Routing

Chaque requête HTTP arrive sur index.php. Ce fichier lit $_GET["action"], appelle controleurPrincipal($action) qui retourne le nom du fichier contrôleur à inclure, puis l'inclut.

3.1 index.php

Point d'entrée unique. Inclut getRacine.php pour définir $racine (chemin absolu), puis controleurPrincipal.php. Selon le paramètre action en GET, il inclut le contrôleur correspondant.

3.2 getRacine.php

Définit la variable globale $racine avec dirname(__FILE__), permettant à tous les fichiers inclus d'utiliser des chemins absolus et d'éviter les erreurs de chemin relatif.

3.3 controleurPrincipal.php

Contient la fonction controleurPrincipal($action) qui maintient un tableau associatif des actions possibles et retourne le fichier contrôleur à charger. Si l'action est inconnue, elle retourne listeRestos.php par défaut.

3.4 Table de routing

Action GET	Contrôleur chargé	Description
`(vide / defaut)`	`listeRestos.php`	Page d'accueil — liste tous les restaurants
`action=liste`	`listeRestos.php`	Liste des restaurants
`action=detail&idR=N`	`detailResto.php`	Fiche détail d'un restaurant (id=N)
`action=recherche`	`rechercheResto.php`	Formulaire de recherche (+ résultats si POST)
`action=connexion`	`connexion.php`	Formulaire de connexion / traitement login
`action=deconnexion`	`deconnexion.php`	Déconnexion et redirection
`action=aimer&idR=N`	`aimer.php`	Ajouter un « j'aime » sur le restaurant N
`action=critiquer&idR=N`	`critiquer.php`	Poster ou supprimer un avis (POST)

4. Couche Contrôleur — Détail des fichiers

4.1 listeRestos.php

Contrôleur par défaut. Charge les modèles bd.resto.php, bd.photo.php, bd.typecuisine.php et bd.critiquer.php. Appelle getLesRestos() pour récupérer tous les restaurants, définit le titre de page et inclut la vue entete + vueListeRestos + pied.

4.2 detailResto.php

Récupère l'idR depuis $_GET, construit le menu burger contextuel (ancres internes : #top, #adresse, #photos, #horaires, #crit). Charge les données du restaurant, ses photos, ses critiques, ses types de cuisine et les « aimers ». Inclut la vue détail.

4.3 rechercheResto.php

Lit le critère de recherche depuis $_GET["critere"] (défaut : "nom"). Récupère les valeurs des formulaires en POST ($nomR, $villeR, $cpR, $voieAdrR, $tabIdTC[]). Selon le critère, appelle la fonction modèle correspondante. Inclut la vue vueListeRestos uniquement si une soumission POST a eu lieu.

Trois modes de recherche sont supportés :

- Par nom : recherche LIKE sur nomR
- Par adresse : recherche LIKE sur voieAdrR, cpR et villeR
- Par type de cuisine : filtrage via tableau de checkboxes et jointure SQL

4.4 connexion.php

Gère trois cas selon la présence ou l'absence de $_POST["mailU"] et $_POST["mdpU"] :

1. Aucune donnée POST → affiche le formulaire vueAuthentification
2. POST valide + login() réussi → affiche vueConfirmation
3. POST invalide ou login() échoué → réaffiche vueAuthentification

4.5 deconnexion.php

Appelle logout() pour effacer la session puis redirige vers l'action connexion via header(). Note : la redirection contient une cible légèrement incorrecte ("connexion.php" au lieu de "connexion").

4.6 aimer.php

Vérifie que l'utilisateur est connecté via isLoggedOn(). Si oui, récupère son email et l'idR en GET, insère un enregistrement dans la table aimer via addAimer(), puis redirige vers la fiche détail du restaurant.

4.7 critiquer.php

Récupère note et commentaire en POST. Si l'utilisateur est connecté, vérifie s'il a déjà une critique pour ce restaurant. Si oui, supprime la critique (delCritiquer). Sinon, en ajoute une nouvelle (addCritiquer) puis redirige.

5. Couche Modèle — Détail des fichiers

5.1 bd.pdo.php — Connexion à la base

Contient la fonction connexionPDO() qui instancie une connexion PDO vers la base MySQL resto sur localhost. Les paramètres de connexion sont :

- Hôte : localhost
- Base : resto
- Utilisateur : root
- Mot de passe : (vide — configuration WampServer locale)
- Encodage : UTF-8 forcé via INIT_COMMAND

Le mode d'erreur est réglé sur ERRMODE_EXCEPTION pour faciliter le débogage.

5.2 authentification.php — Gestion de session

Inclut bd.utilisateur.php. Contient quatre fonctions de gestion de session :

- login() : récupère l'utilisateur BDD, compare le mot de passe via crypt() (sel = hash BDD), initialise $_SESSION
- logout() : efface mailU et mdpU de la session
- isLoggedOn() : vérifie que la session correspond à un utilisateur BDD existant
- getMailULoggedOn() : accesseur pratique pour l'email de l'utilisateur connecté

5.3 Référence complète des fonctions modèle

Fonction	Fichier	Description
`connexionPDO()`	`bd.pdo.php`	Crée et retourne une connexion PDO à la BDD resto
`login($mail, $mdp)`	`authentification.php`	Vérifie les credentials et initialise la session
`logout()`	`authentification.php`	Détruit les variables de session mailU et mdpU
`isLoggedOn()`	`authentification.php`	Retourne true si la session correspond à un utilisateur BDD valide
`getMailULoggedOn()`	`authentification.php`	Retourne l'email de l'utilisateur connecté ou chaîne vide
`getLesRestos()`	`bd.resto.php`	SELECT * FROM resto — tous les restaurants
`getLeRestoByIdR($idR)`	`bd.resto.php`	Retourne un restaurant par son idR
`getLesRestosBynomR($nom)`	`bd.resto.php`	Recherche LIKE sur le nom du restaurant

`getLesRestosByAdresse($voie,$cp,$ville)`	`bd.resto.php`	Recherche LIKE sur rue, code postal et ville
`getLesRestosByTC($tabIdTC)`	`bd.resto.php`	Recherche par tableau d'idTC via INNER JOIN proposer
`getLaPhotosByIdR($idR)`	`bd.photo.php`	Retourne une seule photo (fetch) pour un restaurant
`getLesPhotosByIdR($idR)`	`bd.photo.php`	Retourne toutes les photos (fetchAll) d'un restaurant
`getLesTypesCuisinebyIdR($idR)`	`bd.typecuisine.php`	Types de cuisine d'un restaurant (JOIN proposer)
`getLesTypesCuisine()`	`bd.typecuisine.php`	Tous les types de cuisine disponibles
`getLeUtilisateurByMailU($mail)`	`bd.utilisateur.php`	Retourne un utilisateur par son email
`getAimerById($mailU, $idR)`	`bd.aimer.php`	Retourne les enregistrements aimer pour ce restaurant
`addAimer($mailU, $idR)`	`bd.aimer.php`	INSERT INTO aimer — l'utilisateur aime ce restaurant
`delAimer($mailU, $idR)`	`bd.aimer.php`	DELETE FROM aimer — retirer le j'aime
`getCritiquerByIdR($idR)`	`bd.critiquer.php`	Toutes les critiques d'un restaurant
`getCritiquerById($idR, $mailU)`	`bd.critiquer.php`	Critique d'un utilisateur précis sur un restaurant
`addCritiquer($mailU,$idR,$note,$comm)`	`bd.critiquer.php`	INSERT INTO critiquer (note + commentaire)
`delCritiquer($idR, $mailU)`	`bd.critiquer.php`	DELETE FROM critiquer

6. Couche Vue — Détail des fichiers

Toutes les vues sont des fichiers PHP qui génèrent du HTML. Elles reçoivent les données via les variables définies dans le contrôleur (pas d'accès direct à la BDD ni aux superglobales $_GET / $_POST).

6.1 Structure commune des pages

Chaque page est composée de trois inclusions systématiques depuis le contrôleur :

4. entete.html.php — génère tout le <head>, le <nav> et ouvre le
5. vue spécifique — contenu de la page
6. pied.html.php — ferme les balises ouvertes

6.2 Tableau des vues

Vue	Description
`entete.html.php`	En-tête HTML commun : doctype, importation des CSS, barre de navigation principale, menu contextuel burger
`pied.html.php`	Ferme les balises </body></html>
`vueListeRestos.php`	Boucle sur $lesRestos, affiche une carte par restaurant avec sa photo et ses tags de cuisine
`vueDetailResto.php`	Fiche complète : nom, bouton aimer, étoiles de note, types de cuisine, description, adresse, galerie photos, horaires, formulaire de critique
`vueRechercheResto.php`	Formulaire dynamique selon $critere : champ nom, champs adresse (ville/CP/rue), ou cases à cocher types de cuisine
`vueAuthentification.php`	Formulaire de connexion (email + mot de passe) et lien vers inscription
`vueConfirmation.php`	Page post-connexion affichant l'email et le lien de déconnexion

6.3 entete.html.php

Génère le DOCTYPE, importe les CSS, affiche la barre de navigation principale (Accueil, Liste, Recherche, logo, Connexion/Déconnexion selon isLoggedOn()). Gère également l'affichage conditionnel du menu burger contextuel ($menuBurger) si la variable est définie par le contrôleur.

6.4 vueDetailResto.php

Vue la plus complexe du projet. Affiche dans l'ordre : le nom du restaurant avec bouton aimer (image aime.png ou aimepas.png), les étoiles de notation (boucle fixe 1-5), les types de cuisine en tags, la description, l'adresse, la galerie de photos, les horaires en tableau HTML, puis la liste des critiques. Si aucune critique n'existe pour cet utilisateur, un formulaire est affiché pour en poster une (note par radio bouton + commentaire textarea).

7. Base de données MySQL

La base de données se nomme resto et est exportée dans le fichier base de donnees/resto.sql. Elle contient 8 tables dont 3 tables de liaison.

7.1 Schéma des tables

Table	Clé primaire	Colonnes principales
`utilisateur`	`mailU`	mailU, mdpU (crypt), pseudoU
`resto`	`idR`	nomR, numAdrR, voieAdrR, cpR, villeR, latitudeDegR, longitudeDegR, descR, horairesR
`photo`	`idP`	cheminP, idR (FK resto)
`typecuisine`	`idTC`	libelleTC
`proposer`	`(idR, idTC)`	Association restaurant ↔ type de cuisine
`aimer`	`(idR, mailU)`	Association utilisateur ↔ restaurant aimé
`critiquer`	`(idR, mailU)`	note (int 1-5), commentaire (varchar 4096)
`preferer`	`(mailU, idTC)`	Types de cuisine préférés par utilisateur

7.2 Relations entre les tables

- utilisateur ← aimer → resto : un utilisateur peut aimer plusieurs restaurants
- utilisateur ← critiquer → resto : un utilisateur peut critiquer un restaurant (1 critique max par couple)
- utilisateur ← preferer → typecuisine : préférences culinaires de l'utilisateur
- resto ← proposer → typecuisine : un restaurant peut proposer plusieurs types de cuisine
- resto ← photo : photos associées à un restaurant (plusieurs par restaurant)

7.3 Données de test

Le dump SQL inclut un utilisateur de test : test@bts.sio / mdpU : seSzpoUAQgIl. (hashé via crypt()). 11 restaurants sont présents principalement à Bordeaux et Bayonne. Les tables aimer, critiquer et preferer contiennent des données exemples associées à ce compte de test.

7.4 Types de cuisine disponibles

11 types de cuisine sont définis : sud ouest, japonaise, orientale, fastfood, végétarienne, vegan, crêpe, sandwich, tartes, viande, grillade.

8. Feuilles de style CSS

Le projet utilise quatre fichiers CSS importés dans entete.html.php. La police Poppins est importée depuis Google Fonts.

8.1 Variables CSS (base.css)

Les variables définies dans :root permettent une personnalisation centralisée :

- --largeurPage: 900px — largeur du contenu principal
- --hauteurMenuGene: 60px — hauteur de la barre de navigation
- --coulA1: #3B5998 — bleu principal (liens)
- --coulB1: #1ABC9C — vert accent (boutons, titres)
- --coulB2: #16A085 — vert foncé (hover)
- --coulC: #F39C12 — orange (alertes)
- --fondC: #F5F7FA — gris clair de fond

8.2 Tableau des fichiers CSS

Fichier	Contenu principal
`base.css`	Variables CSS (couleurs, dimensions), mise en page générale, nav fixe, menu burger, animations de transition
`corps.css`	Cartes restaurant (#card, .photoCard), galerie photos, tags de cuisine (#tagFood, li.tag)
`form.css`	Champs input/password/select/textarea, labels, boutons submit, hover effects
`menu.css`	Réécriture complète du nav, menu burger responsive, media query @768px, menu latéral animé

8.3 Composants CSS notables

Les cartes restaurant (div.card) font 220×300px, flottent à gauche, ont un box-shadow et un effet scale(1.05) au survol. La galerie photos utilise CSS Grid avec auto-fill. Le menu burger est géré en CSS pur par l'effet :hover sur #bouton (sans JavaScript). La responsivité (@media max-width: 768px) est définie dans menu.css.

9. Ressources statiques

9.1 Images UI (site/images/)

- logo.png — logo principal (122 Ko)
- logoBarre.png — logo pour la barre de nav (14 Ko)
- aime.png / aimepas.png — icônes état j'aime
- like.png — icône étoile de notation
- neutre.png — icône note neutre
- coche.png — icône validation
- profil.png — icône utilisateur dans le menu
- rechercher.png — icône recherche

9.2 Photos restaurants (site/photos/)

12 photos JPEG sont fournies pour les 11 restaurants de démonstration : agadir.jpg, auberge.jpg, barDuCharcutier.jpg, barDuMarche.jpg, cidrerieDuFronton.jpg, entrepote.jpg, laTableDePottoka.jpg, leBistrotSainteCluque.jpg, rotisserieDuRoyLeon.jpg, sapporo.jpg, sapporo2.jpg, trinquetModerne.jpg.

10. Bugs identifiés et points d'amélioration

L'analyse du code source révèle plusieurs bugs, dont deux critiques qui empêchent le bon fonctionnement de certaines fonctionnalités.

10.1 Tableau des bugs

Fichier	Niveau	Description
`bd.critiquer.php — addCritiquer()`	□ Critique	Requête INSERT INTO critiquer VALUES(?, ?, ?, ?) WHERE ... — syntaxe SQL invalide (INSERT n'accepte pas de clause WHERE). La critique ne sera jamais insérée.
`controleur/critiquer.php`	□ Critique	Appel addCritiquer($mailU, $idR, $note, $critiquer) — ordre des paramètres inversé par rapport à la signature de la fonction addCritiquer($idR, $mailU, $note, $commentaire).
`controleur/deconnexion.php`	□ Mineur	header("location:./?action=connexion.php") — l'action devrait être "connexion" sans ".php".
`vue/vueDetailResto.php`	□ Mineur	Guillemets non fermés dans les attributs alt des balises <img> de la section aimer.
`bd.aimer.php — getAimerById()`	□ Mineur	Le paramètre $mailU est reçu mais n'est pas utilisé dans la requête SQL (filtre uniquement sur idR). La fonction ne vérifie pas si l'utilisateur courant a aimé ce restaurant.

10.2 Points d'amélioration suggérés

- Sécurité : le mot de passe est haché avec crypt() (DES, déprécié). Migrer vers password_hash() / password_verify() (bcrypt).
- Prévention CSRF : aucun token anti-CSRF sur les formulaires de critique et de connexion.
- Injections : les requêtes utilisent PDO avec bindValue() — c'est correct. La construction dynamique de la clause IN dans getLesRestosByTC() concatène directement les idTC sans validation du type entier.
- La table preferer (préférences utilisateur) est définie en BDD mais aucune fonction modèle ni aucun contrôleur ne l'exploite pour personnaliser les résultats de recherche.
- Pagination absente : getLesRestos() retourne tous les restaurants sans limite.
- Inscription : le lien vers ?action=inscription existe dans la vue mais aucun contrôleur inscription n'est déclaré dans controleurPrincipal.php.

11. Guide d'installation

11.1 Prérequis

- WampServer (ou LAMP/MAMP équivalent) avec PHP ≥ 5.6 et MySQL 5.7
- PHPMyAdmin pour importer le dump SQL

11.2 Étapes

7. Copier le dossier site/ dans le répertoire www/ de WampServer (ex: www/r3st0/)
8. Ouvrir PHPMyAdmin, créer une base de données nommée resto
9. Importer le fichier base de donnees/resto.sql dans cette base
10. Vérifier les paramètres de connexion dans site/modele/bd.pdo.php (login root, mdp vide)
11. Accéder au site via http://localhost/r3st0/
12. Se connecter avec le compte de test : test@bts.sio / (mot de passe dans le dump)

Annexe — Résumé des fichiers

Le projet contient 61 fichiers au total répartis comme suit :

- 1 fichier SQL (base de données complète)
- 8 fichiers PHP Contrôleur
- 8 fichiers PHP Modèle
- 7 fichiers PHP Vue
- 4 fichiers CSS
- 1 fichier utilitaire (getRacine.php)
- 1 point d'entrée (index.php)
- 9 images UI
- 12 photos de restaurants

— Fin de la documentation —